

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №1

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Простые структуры данных"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М.А.

Принял:

к.т.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение и повторение простых структур данных

Лабораторное задание

Задание 1: написать программу, вычисляющую разницу между максимальным и минимальным элементами массива.

Задание 2: написать программу, реализующую инициализацию массива случайными числами.

Задание 3: написать программу, реализующую создание массива произвольного размера, вводимого с клавиатуры.

Задание 4: написать программу, вычисляющую сумму значений в каждом столбце (или строке) двумерного массива.

Задание 5: написать программу, осуществляющую поиск среди структуры `student` с заданными параметрами (фамилией, именем и возрастом).

Пояснительный текст к программе

Задание 1

Программа создает массив из 5 элементов и заполняет его цифрами от 0 до 9. Программа находит минимальное и максимальное значение в массиве и вычисляет их разницу и выводит результат.

Задание 2

Программа создает массив из 5 элементов и заполняет его случайными значениями от 0 до 100 с использованием функции `srand()`.

Задание 3

Программа запрашивает размер массива, затем выводит полученный массив на экран.

Задание 4

Программа вычисляет сумму значений в каждом столбце (или строке) двумерного массива.

Задание 5

Программа содержит информацию о 3 студентах. Используется метод поиска по имени и фамилии студента, если введены неправильные данные, выводится на экран предупреждение о том, что такого студента нет.

Результат программы

```
min: 200  
max: 11000  
difference: 10800
```

1 Рис. - Результат работы программы 1-zadan.cpp

```
Сгенерированный массив:  
-6171 10418 -5463
```

2 Рис. - Результат работы программы 2-zadan.cpp

```
Введите размер массива: 3  
Введите 3 элементов массива:  
Элемент 1: 20  
Элемент 2: 12  
Элемент 3: 1  
Введенный массив:  
20 12 1
```

3 Рис. - Результат работы программы 3-zadan.cpp

```
Сгенерированная матрица (3 x 5):  
15  52  1  13  2  
97  16 32  9  16  
14  35 18  25 32  
Суммы по столбцам:  
Столбец 1: 126  
Столбец 2: 103  
Столбец 3: 51  
Столбец 4: 47  
Столбец 5: 50
```

4 Рис. - Результат работы программы 4-zadan.cpp

```
Введите данные для студента 1:  
Фамилия (минимум 3 символа): а  
Ошибка: фамилия должна содержать минимум 3 символа!  
Фамилия (минимум 3 символа): aa  
Ошибка: фамилия должна содержать минимум 3 символа!
```

5 Рис. - Результат работы программы 5-zadan.cpp

```
Введите фамилию для поиска: aaa  
  
Результаты поиска:  
Найден: aaa asds, факультет asdasdf, № 2334
```

6 Рис. - Результат работы программы 5-zadan.cpp

Листинг

Файл lbb1.cpp

```
#define _CRT_SECURE_NO_WARNINGS  
#include <stdio.h>  
#include "stdafx.h"  
#include <stdlib.h>  
#include <string.h>  
#include <locale.h>  
#include <time.h>  
  
void task1();  
void task2();  
void task3();  
void task4();  
void task5();  
  
int main() {  
    setlocale(LC_ALL, "rus");  
    int choice;  
  
    do {  
        printf("\n=== МЕНЮ ВЫБОРА ЗАДАНИЯ ===\n");  
        printf("1. Задание 1 - Разница между max и min элементами массива\n");  
        printf("2. Задание 2 - Генерация массива случайных чисел\n");  
        printf("3. Задание 3 - Ввод и вывод массива\n");  
        printf("4. Задание 4 - Суммы по столбцам матрицы\n");  
        printf("5. Задание 5 - Поиск студентов\n");  
        printf("0. Выход\n");  
        printf("Выберите номер задания: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                task1();  
                break;  
            case 2:  
                task2();  
                break;  
            case 3:  
                task3();  
                break;  
            case 4:  
                task4();  
                break;  
            case 5:  
                task5();  
                break;  
            case 0:  
                return 0;  
            default:  
                printf("Неверный номер задания.\n");  
        }  
    } while (choice != 0);  
}
```

```

        task3();
        break;
    case 4:
        task4();
        break;
    case 5:
        task5();
        break;
    case 0:
        printf("Выход из программы...\n");
        break;
    default:
        printf("Неверный выбор! Попробуйте снова.\n");
    }

    if (choice != 0) {
        printf("\nНажмите Enter для продолжения...");
        while (getchar() != '\n');
        getchar();
    }
} while (choice != 0);

return 0;
}

```

// Задание 1: Разница между максимальным и минимальным элементами массива

```

void task1() {
    printf("\n=== ЗАДАНИЕ 1 ===\n");
    int n, i;
    int* arr;
    int min, max, difference;

    printf("Введите размер массива: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Ошибка: размер массива должен быть положительным!\n");
        return;
    }

    arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("Ошибка выделения памяти!\n");
        return;
    }

    printf("Введите %d элементов массива:\n", n);
    for (i = 0; i < n; i++) {
        printf("Элемент %d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    min = arr[0];
    max = arr[0];

    for (i = 1; i < n; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    difference = max - min;
}

```

```

printf("min: %d\n", min);
printf("max: %d\n", max);
printf("difference: %d\n", difference);

free(arr);
}

// Задание 2: Инициализация массива случайными числами
void task2() {
    printf("\n=== ЗАДАНИЕ 2 ===\n");
    int n, i, lower, upper;
    int* arr;

    srand(time(NULL));

    printf("Введите размер массива: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Ошибка: размер массива должен быть положительным!\n");
        return;
    }

    srand(time(NULL));

    arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("Ошибка выделения памяти!\n");
        return;
    }

    printf("Введите нижнюю границу диапазона: ");
    scanf("%d", &lower);
    printf("Введите верхнюю границу диапазона: ");
    scanf("%d", &upper);

    if (lower > upper) {
        printf("Ошибка: нижняя граница не может быть больше верхней!\n");
        free(arr);
        return;
    }

    printf("Сгенерированный массив:\n");
    for (i = 0; i < n; i++) {
        arr[i] = lower + rand() % (upper - lower + 1);
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);
}

// Задание 3: Создание массива произвольного размера
void task3() {
    printf("\n=== ЗАДАНИЕ 3 ===\n");
    int n, i;
    int* arr;

    printf("Введите размер массива: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Ошибка: размер массива должен быть положительным!\n");
    }
}

```

```

    return;
}

arr = (int*)malloc(n * sizeof(int));
if (arr == NULL) {
    printf("Ошибка выделения памяти!\n");
    return;
}

printf("Введите %d элементов массива:\n", n);
for (i = 0; i < n; i++) {
    printf("Элемент %d: ", i + 1);
    scanf("%d", &arr[i]);
}

printf("Введенный массив:\n");
for (i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

free(arr);
}

// Задание 4: Сумма значений в каждом столбце двумерного массива
void task4() {
    printf("\n=== ЗАДАНИЕ 4 ===\n");
    int rows, cols, i, j;
    int** arr;
    int* col_sums;

    printf("Введите количество строк: ");
    scanf("%d", &rows);
    printf("Введите количество столбцов: ");
    scanf("%d", &cols);

    if (rows <= 0 || cols <= 0) {
        printf("Ошибка: размеры матрицы должны быть положительными!\n");
        return;
    }

    srand(time(NULL));

    arr = (int**)malloc(rows * sizeof(int*));
    for (i = 0; i < rows; i++) {
        arr[i] = (int*)malloc(cols * sizeof(int));
    }

    col_sums = (int*)calloc(cols, sizeof(int));

    printf("Заполнение матрицы случайными числами\n");
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            arr[i][j] = rand() % 100;
            col_sums[j] += arr[i][j];
        }
    }

    printf("\nСгенерированная матрица (%d x %d):\n", rows, cols);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            printf("%4d ", arr[i][j]);

```

```

    }
    printf("\n");
}

printf("\nСуммы по столбцам:\n");
for (j = 0; j < cols; j++) {
    printf("Столбец %d: %d\n", j + 1, col_sums[j]);
}

for (i = 0; i < rows; i++) {
    free(arr[i]);
}
free(arr);
free(col_sums);
}

// Задание 5: Поиск среди структур student
void task5() {
    printf("\n=== ЗАДАНИЕ 5 ===\n");

    struct student {
        char surname[50];
        char name[50];
        char faculty[50];
        int record_book;
    };

    int n, i, choice, found;
    char search_str[50];
    int search_int;

    printf("Введите количество студентов: ");
    scanf("%d", &n);
    while (getchar() != '\n');

    if (n <= 0) {
        printf("Ошибка: количество студентов должно быть положительным!\n");
        return;
    }

    struct student* stud = (struct student*)malloc(n * sizeof(struct student));
    if (stud == NULL) {
        printf("Ошибка выделения памяти!\n");
        return;
    }

    for (i = 0; i < n; i++) {
        printf("\nВведите данные для студента %d:\n", i + 1);

        do {
            printf("Фамилия (минимум 3 символа): ");
            fgets(stud[i].surname, 50, stdin);

            stud[i].surname[strcspn(stud[i].surname, "\n")] = '\0';

            if (strlen(stud[i].surname) < 3) {
                printf("Ошибка: фамилия должна содержать минимум 3 символа!\n",
                    (int)strlen(stud[i].surname));
            }
        }

        while (strlen(stud[i].surname) < 3);
    }
}

```



```

printf("Имя: ");
fgets(stud[i].name, 50, stdin);
stud[i].name[strcspn(stud[i].name, "\n")] = '\0';

printf("Факультет: ");
fgets(stud[i].faculty, 50, stdin);
stud[i].faculty[strcspn(stud[i].faculty, "\n")] = '\0';

printf("Номер зачетной книжки: ");
scanf("%d", &stud[i].record_book);
while (getchar() != '\n');
}

printf("\nПо какому полю выполнить поиск?\n");
printf("1 - Фамилия\n");
printf("2 - Имя\n");
printf("3 - Факультет\n");
printf("4 - Номер зачетной книжки\n");
printf("Ваш выбор: ");
scanf("%d", &choice);
while (getchar() != '\n');

found = 0;

switch (choice) {
case 1:
    printf("Введите фамилию для поиска: ");
    fgets(search_str, 50, stdin);
    search_str[strcspn(search_str, "\n")] = '\0';
    printf("\nРезультаты поиска:\n");
    for (i = 0; i < n; i++) {
        if (strcmp(stud[i].surname, search_str) == 0) {
            printf("Найден: %s %s, факультет %s, № %d\n",
                stud[i].surname, stud[i].name, stud[i].faculty, stud[i].record_book);
            found = 1;
        }
    }
    break;

case 2:
    printf("Введите имя для поиска: ");
    fgets(search_str, 50, stdin);
    search_str[strcspn(search_str, "\n")] = '\0';
    printf("\nРезультаты поиска:\n");
    for (i = 0; i < n; i++) {
        if (strcmp(stud[i].name, search_str) == 0) {
            printf("Найден: %s %s, факультет %s, № %d\n",
                stud[i].surname, stud[i].name, stud[i].faculty, stud[i].record_book);
            found = 1;
        }
    }
    break;

case 3:
    printf("Введите факультет для поиска: ");
    fgets(search_str, 50, stdin);
    search_str[strcspn(search_str, "\n")] = '\0';
    printf("\nРезультаты поиска:\n");
    for (i = 0; i < n; i++) {
        if (strcmp(stud[i].faculty, search_str) == 0) {
            printf("Найден: %s %s, факультет %s, № %d\n",
                stud[i].surname, stud[i].name, stud[i].faculty, stud[i].record_book);
            found = 1;
        }
    }
}

```

```

    }
}
break;

case 4:
    printf("Введите номер зачетной книжки для поиска: ");
    scanf("%d", &search_int);
    while (getchar() != '\n');
    printf("\nРезультаты поиска:\n");
    for (i = 0; i < n; i++) {
        if (stud[i].record_book == search_int) {
            printf("Найден: %s %s, факультет %s, № %d\n",
                stud[i].surname, stud[i].name, stud[i].faculty, stud[i].record_book);
            found = 1;
        }
    }
    break;

default:
    printf("Неверный выбор!\n");
    free(stud);
    return;
}

if (!found) {
    printf("Студент с указанными параметрами не найден.\n");
}

printf("\n=== ВСЕ СТУДЕНТЫ ===\n");
for (i = 0; i < n; i++) {
    printf("Студент %d: %s %s, %s, №%d\n",
        i + 1, stud[i].surname, stud[i].name, stud[i].faculty, stud[i].record_book);
}

free(stud);
}

```

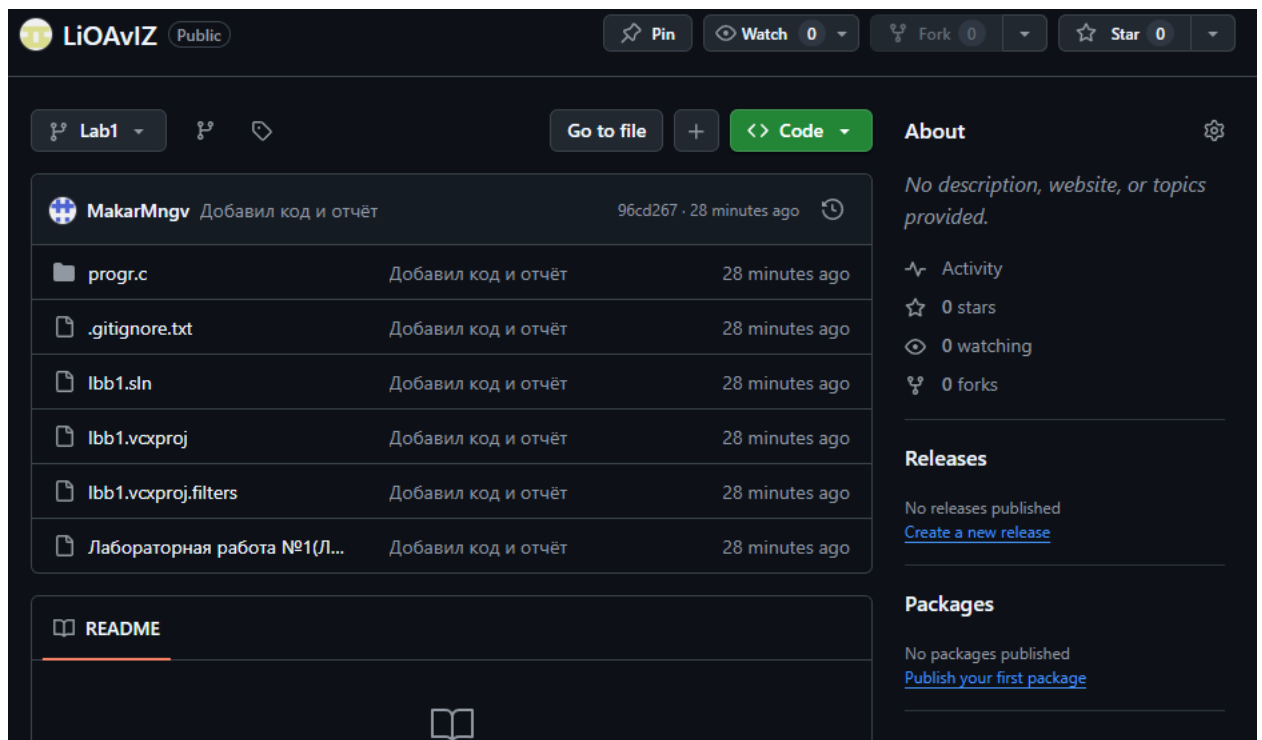
Загрузка программы в GitHub

```
gdffh@DESKTOP-HDP8KVQ MINGW64 ~/OneDrive/Рабочий стол/gitnew/LiOAvIZ (master)
$ git push -u origin master
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 356.45 KiB | 29.70 MiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/MakarMn/LiOAvIZ.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

gdffh@DESKTOP-HDP8KVQ MINGW64 ~/OneDrive/Рабочий стол/gitnew/LiOAvIZ (master)
$ git branch -m Lab1

gdffh@DESKTOP-HDP8KVQ MINGW64 ~/OneDrive/Рабочий стол/gitnew/LiOAvIZ (Lab1)
$ git push -u origin Lab1
```

7 рис. – Загрузка локального репозитория в GitHub



8 рис. – Результат загрузки репозитория в GitHub на новой ветке

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №1. В процессе выполнения работы были использованы знания о простейших структурах данных.

Ссылка на удаленный репозиторийGitHub:

<https://github.com/MakarMn/LiOAvIZ.git>